AutoGrader

A Rule-Driven Evaluator for C Programs using open weight LLM-Models and modularized for expert analyzers on different aspects of evaluation metrics for independent and consistent evaluation.

Balaji Vara Prasad Dega BALAJI_2611AI05@IITP.AC.IN
2611AI05

Uddipta Arun Paul UDDIPTA_2611AI06@IITP.AC.IN
2611AI06

Suva Pratyush Tripathy SUVA_2611AI14@IITP.AC.IN
2611AI14

Asadhi Vamshi Raj ASADHI_2611MC14@IITP.AC.IN
2611MC14

Editor: Balaji Vara Prasad Dega

Abstract

Manual grading of C programs is slow, inconsistent, and hard to scale. This report presents **AutoGrader**, an agentic, rule-driven system that evaluates student C submissions unattended and returns one reproducible grade with actionable feedback. Given a submission and an instructor-authored rules file, the system compiles and runs the program in a sandbox with fixed time and memory limits, evaluates it against every metric, and produces both a JSON report and a student-facing text report.

A coordinator agent first performs a sanity sweep (compile check, smoke run, static checks). If the program crashes or never halts, execution-based analyzers are skipped and a partial report is emitted. Otherwise, independent analyzers assess correctness, compiler warnings, structure, modularity, edge cases, empirical time complexity, memory safety (ASan/UBSan), readability (a local LLM via Ollama[1] or any other open-source/open-weight LLMs that are more compatible with the local system configuration and capability), and originality (similarity scoring). Metrics, thresholds, and weights are stored in an external JSON file, so instructors can change them without editing code.

Reproducibility is ensured through fixed test sets, hashed rules, pinned tool versions, and LLM calls with temperature zero and a fixed seed. For incorrect submissions, the agent generates counter-test cases covering corner cases such as overflow, reporting the failing input, expected output, and an explanation. The report also discusses real-life usefulness and concrete limitations of the system.

1. Introduction

1.1 Idea

We will write a Python program that takes a student's C file and the teacher's rules file, runs the code safely, checks it on every metric, and gives one final grade with feedback. Same code and same rules should always give the same grade.

1.2 Problem Statement

The task is to design and implement an intelligent agent that grades student C programs automatically according to rules supplied by the instructor. The agent takes two inputs: a student's C source file and a rules file describing the metrics, thresholds, and weights to apply. It must then:

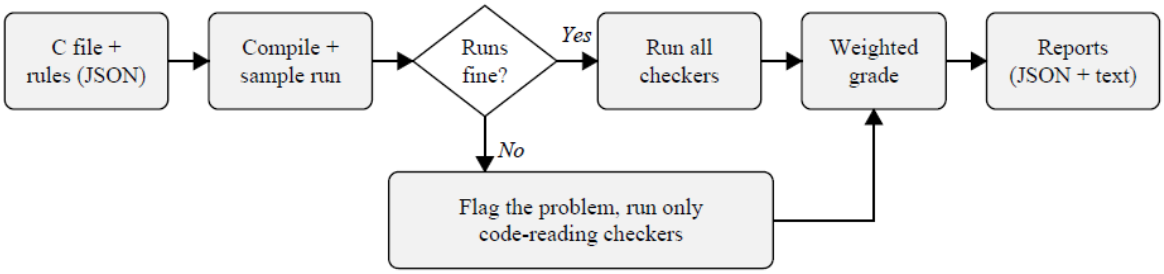


Figure 1: Overall flow of the grader.

1. compile and run the submission safely under fixed time and memory limits, and survive programs that crash or never halt;
2. evaluate the submission against every metric in the rules, using a dedicated analyzer for each;
3. combine the analyzer results into one overall grade using the specified weights;
4. produce feedback that explains where and why the program falls short, including a failing input with the expected output beside it for incorrect submissions;
5. flag suspected copying from other submissions, online resources, or LLM-generated code.

A simple example illustrates the setting. The problem is to read two integers on one line and print their sum followed by a newline. A student submits a program that works on typical inputs, but it places all logic in `main()`, uses single-letter variable names, has no comments, and silently overflows when given `INT_MAX + 1`. A useful grader should detect these weaknesses and score each one under the appropriate metric. It should then report the failing input `2147483647 1` together with the expected output and an explanation of the signed overflow. It should also suggest concrete improvements, such as extracting an `add(int, int)` helper, renaming variables, and using a wider integer type.

Three constraints make the problem non-trivial:

- **Safety:** student code is untrusted and may loop forever, crash, or consume excessive memory, so execution must be sandboxed and bounded.
- **Configurability:** the set of metrics is open. Correctness, time efficiency, code structure, and originality are examples, and the rules may add others. Metrics, thresholds, and weights must therefore live outside the agent's source code.
- **Determinism:** an automated grade is only trustworthy if it can be reproduced, so every stochastic or time-dependent component must be controlled.

1.3 Key design choices

The objectives of this project are as follows.

1. **Rules in a JSON file :** Limits, metrics, weights and thresholds live outside the code. Adding a metric means adding one checker and one line in the JSON
2. **Safe running :** Each run happens in a temp folder with CPU, memory and time limits. Crashes and infinite loops are noted in the report instead of stopping the grader.

3. **Same output for every checker :** A score from 0 to 1, the evidence, and one line of feedback.

4. **Final grade :** = 100 x sum of (weight x score).

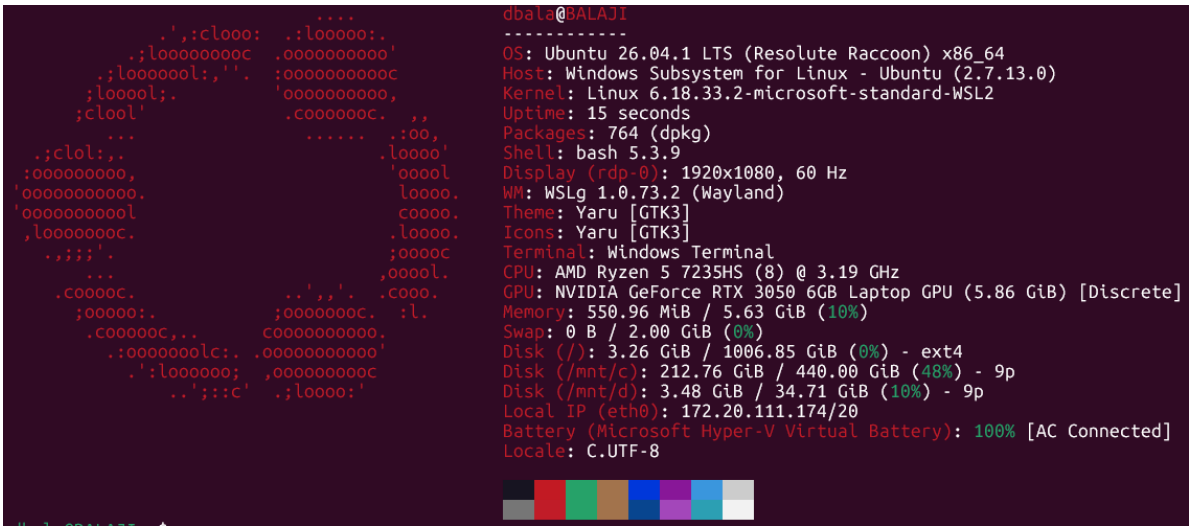


Figure 2: working system configuration fetched using FastFetch package through Ubuntu terminal.

1.4 Checkers

Metric	**How we will check it**
Correctness	Run the hidden test cases and compare outputs.
Syntax	Compile with the given gcc flags and count warnings.
Structure, modularity	lizard and cppcheck for function count, length and complexity.
Edge cases	Tricky inputs (0, negatives, INT_MAX, extra spaces, empty input) and random ones, compared with the teacher's correct solution (Figure 2).
Complexity	Time the program on growing input sizes and see how fast the time grows.
Memory safety	ASan, UBSan and Valgrind for leaks and bad memory access.
Readability	Local LLM (Ollama) at temperature 0 with a fixed seed. Answers are saved and reused.
Originality	Compare code with other submissions and with LLM/online solutions after renaming variables. High similarity is flagged for the teacher.

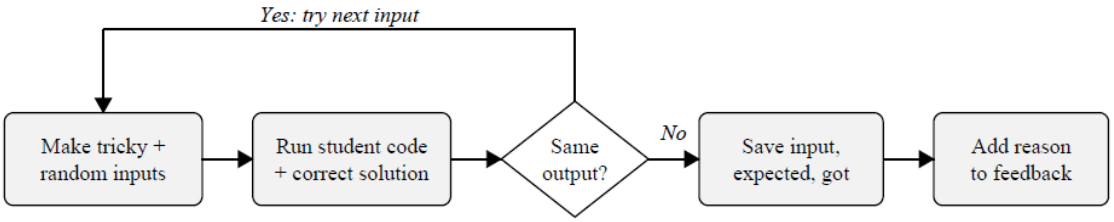


Figure 3: How failing test cases are found.

2. Distribution of Work Among Group Members

2.1 Work Allocation Plan

Member	Work
Balaji Vara Prasad Dega	Main controller, rules loading, safe running, final grade
Suva Pratyush Tripathy	Test runner, edge-case generator, complexity check
Uddipta Arun Paul	Compiler, structure and memory-safety checks
Asadhi Vamshi Raj	LLM review, similarity check, reports, README, video

The project is divided into four parts. Each part has a clear owner and a defined interface (the JSON analyzer contract: score, evidence, feedback), so members can work in parallel and integrate later.

2.2 Coordination and Integration

- All analyzers follow one agreed interface: the input is the source file, the binary and the params slice, and the output is a score in $[0, 1]$, the evidence and a feedback string.
- Code is kept in a shared GitHub repository, with one folder per analyzer. Another member reviews each pull request before it is merged.
- The group meets briefly twice a week, and Member 1 integrates the analyzers into the coordinator as they become ready.
- Every member reviews and can explain the whole codebase, since each member must own the code and the report.

3. Future Plan and Timeline

The first two weeks focus on learning the problem and the tools. The last two weeks focus on implementation, integration and testing.

Week	Phase	Planned activities
Week 1-2	Learning	Study the problem statement and the sample rules file. Learning and understanding the tools to be used for the project. Deciding the work split and work flow.
Week 3	Implementation	*Part 1:* sandbox, first pass, rules loader and coordinator skeleton. *Part 2:* `io_tests`, `edge_case_runner` and `sanitizer_run`. *Part 3:* `compiler_warnings`, `static_analysis` and `function_split`. *Part 4:* test sets, reference solutions and the `similarity_flag` analyzer.
Week 4	Integration and testing	*Part 1:* collation, JSON and text reports, parallel dispatch. *Part 2:* `complexity_estimator` and optional profiling. *Part 3:* `llm_review`. *Part 4:* counter-test generation, reproducibility tests on the sample submissions, limitations section, README, final report and video. *All members:* integrate, fix bugs and rehearse the demo.

Table 1: Four-week implementation timeline.

References

[1] Francisco Marcondes et al. “Using Ollama”. In: Feb. 2025, pp. 23–35. ISBN: 978-3-031-76630-5. DOI: 10.1007/978-3-031-76631-2_3.